CS 5112 Fall 2025 HW2: *Dynamic Programming, Randomization*

Name: Anirudha Hadagali
NetID: ah2495

Due: October 1, 11:59 PM ET

Collaborators: Fill Here

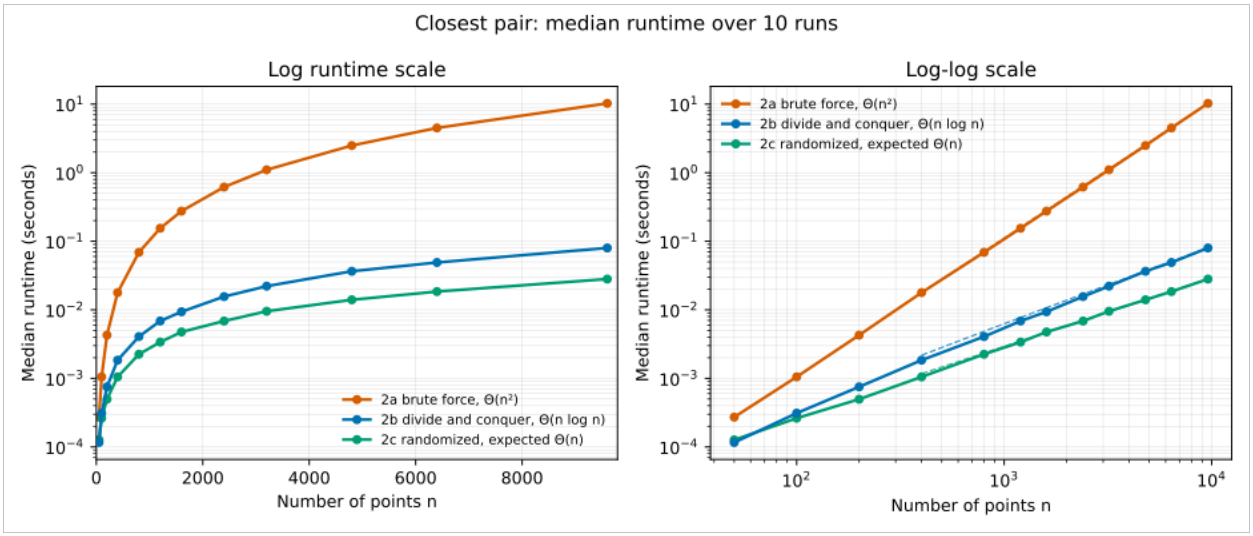
Problem 1.

(a) (*10 points*) ()

Answer: The dynamic programming sub problem that I attempted to solve was: “if I pick this number, will I have a higher sum, or if I skip it and pick its neighbor?” Algorithm Design: In my code, the recursive helper(xs, n, memo) function computes the best sum using only the first n elements. At each call it compares the two options I listed above: a. Pick: add current number to the sum, and then skip its neighbor b. Skip: skip the current number, go to its neighbor The memo fills from small to large, even though the first call is f(7). The recursion goes all the way down before anything gets stored. Memo hits are where the speedup comes from. Without the memo, each of those would start a whole new chain of recursive calls, which makes the plain recursion exponential. With the memo, each f(n) is computed once, so the total work is O(n)

(b) (*10 points*) ()

Answer: The forbidden pairs are the edges, meaning nodes that not directly connected. At each node you either pick v or you don’t: a. Pick v: none of its children can be used, so you add v’s value to the best answers for its grandchildren. b. Don’t pick v: add up the best answers for its children The algorithm is recursion with memoization on the tree, the same idea as 1a but applied to subtrees instead of list prefixes. Every node that shows up in some child list has a parent, so the root is the node that never appears as a child. For each node v, recursive helper computes the best non-adjacent sum within v’s subtree by comparing the two options. Each node’s answer is saved in memo, which saves on not needing to compute



Problem 2.

(a) (*5 points*) ()

Answer: *None*

(b) (*5 points*) ()

Answer: *None*

(c) (*10 points*) ()

Answer: *None*

(d) (*5 points*) ()

Answer: Brute force grows as n**2, divide-and-conquer as n log n, and the randomized grid as n. As predicted, the randomized grid is fast, but the n log n approach from 2b keeps on pace, and is just a bit less efficient. The n**2 approach is exponential and increases rapidly as n increases.

* used an AI assistant for running test cases for n, and plotting the results

n	2a (ms)	2b (ms)	2c (ms)
50	0.272	0.116	0.127
100	1.05	0.311	0.263
200	4.28	0.753	0.497
400	17.8	1.84	1.05
800	69.0	4.07	2.25
1,200	155	6.86	3.38
1,600	275	9.37	4.76
2,400	618	15.6	6.86
3,200	1,102	22.1	9.54
4,800	2,491	36.5	14.0
6,400	4,501	49.0	18.5
9,600	10,269	79.9	28.1

Problem 3.

(a) (*6 points*) ()

Answer: The LinkedList class has a time complexity of O(n). O(n) is the worst case for both insert and search, where n is the number of nodes already in the list. The insert has 3 scenarios: 1. Empty list 2. New node's value is less than the root first node 3. New node needs to get inserted in the middle or at the end

Start at the root and follow 'next' pointers one node at a time. If a node's value equals x, return that node. If the walk falls off the end of the list, x isn't present, so return None

(b) (*6 points*) ()

Answer: Skip Lists was a really interesting concept as it was a continuation of linked lists. Linked lists alawys gave me trouble when I first learned it, but now I can see the benefits of using skip lists. To build the algorithm, I had to build 2 helper methods. One to find the predecessors of all the nodes that I had already visited. Another method to randomly find the level of the new node we would be inserting.

find predecessors(x) is a helper that does the walk search and insert share. It starts at the sentinel on the top level. On each level it moves right while the next value is less than x, saves the node where it stopped, then drops down a level. It returns one saved node per level.

random level keeps calling 'heads', until it gets a 'tails', or if we hit the max number of consecutive heads in a row.

search(x): calls the helper. The only place x can be is right after the saved node on the bottom level, so it checks that one node.

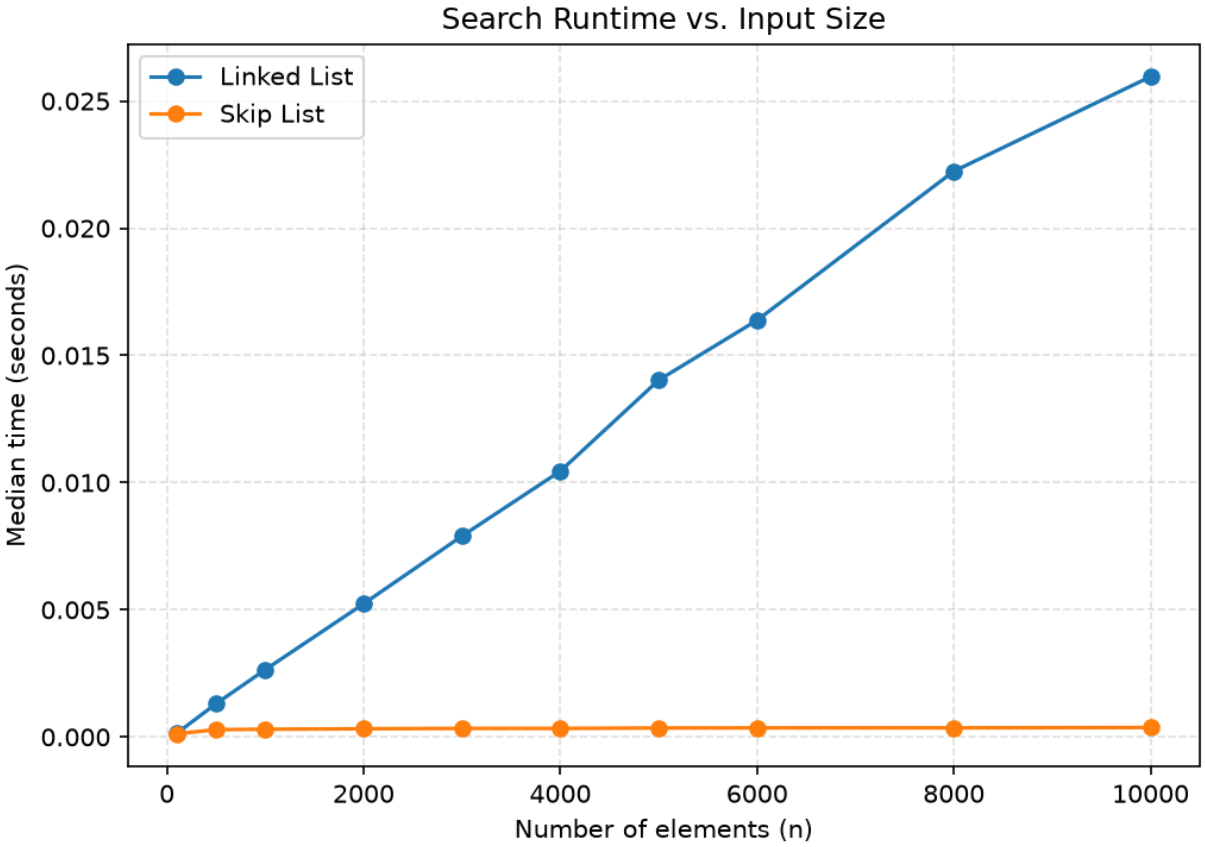
insert(x): calls the helper and does nothing if x is already there. Otherwise it picks a random height and creates one node with that many pointers. On each level it uses the same two-line splice as the linked list, with next[level] instead of next. Then it sets root to sentinel.next[0]. So it shares a lot of functionality with linked lists in that way.

delete(x): the reverse of insert. On each level the node appears on, it points the saved node past the deleted node.

(c) (*3 points*) ()

Answer:

With the help of an AI assistant, for each of 10 sizes, n = 100, 500, 1000, 2000, 3000, 4000, 5000, 6000, 8000 and 10000:


Search Runtime vs. Input Size
Linked List
Skip List
Median time (seconds)
0.025
0.020
0.015
0.010
0.005
0.000
0
2000
4000
6000
8000
10000
Number of elements (n)

I inserted n distinct random integers from the range 0 to 10n. I timed 200 searches for values that are in the list. I repeated this 10 times and plotted the median time.

By viewing the graph, I can see that the Linked Lists' search time grew linearly. If there are 10x elements, it will take 10x time. This fits the O(n) time complexity .

The Skip lists however, it's search time barely grows. Increasing elements by 10 or 100 fold only increases the search time by franctions of a second. This growth fits the expected O(logn) time.

(d) (*2 points*) ()

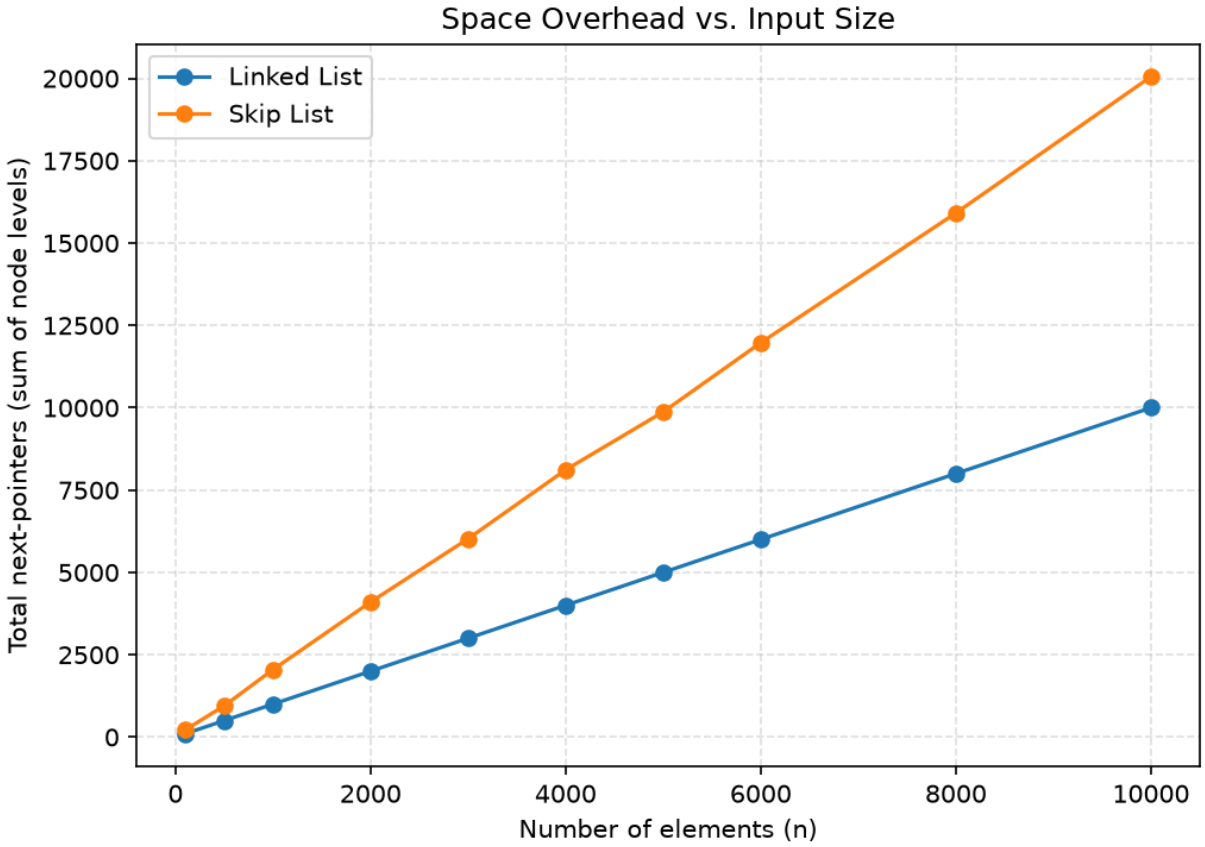
Answer: For Linked Lists, there are n nodes, each storing one value and one next pointer. The total is n pointers, so the overhead is O(n).

For Skip lists, The expected total number of pointers is therefore (n / (1 - p)) + max level, where the second term counts the sentinel's pointers. That is O(n)

In the worst case, every node's coin flips reach the cap, so every node has max level pointers. The total is n · max level, which is O(n · max level)

(e) (*3 points*) ()

Answer: Both structures use space that grows linearly with n. The linked list uses exactly n pointers. The skip list uses 2048 pointers at n = 1000 and 20,053 at n = 10,000, a ratio of about 2.0. The skip list's speedup in (c) comes from roughly doubling memory use.


Space Overhead vs. Input Size
Linked List
Skip List
Total next-pointers (sum of node levels)
Number of elements (n)
0
2500
5000
7500
10000
12500
15000
17500
20000
0
2000
4000
6000
8000
10000